2026 네트워크 게임 프로그래밍
Term 프로젝트 추진 계획서



2022182006 김성찬 2022182010 김정민 2022182037 정태연

한국공학대학교
TECH UNIVERSITY OF KOREA

목차

1. 개요 2

2. 애플리케이션 기획 2

2.1 게임 진행 방식 및 월드 구성 2

2.2 플레이어 이동 및 조작 2

2.3 오브젝트 상호작용 3

2.4 인벤토리 및 도구 선택 4

2.5 농사 및 작물 성장 4

2.6 상점 및 재화 관리 5

2.7 탈출 및 게임 종료 5

2.8 까마귀(추가요소) 6

2.9 주요 게임 화면 6

3. High-Level Design 7

3.1 프로그램 흐름 7

4. Low-Level Design 8

4.1 클래스 설계 8

4.2 프로토콜 설계 9

5. 역할 분담 및 개발 일정 10

5.1 팀원별 역할 분담 10

5.2 개발 일정 10

1. 개요

- 2025년도 2학기 컴퓨터 그래픽스 기말 프로젝트「MiniFarm」기반(김정민 개발)
- 외딴 섬을 배경으로 하는 3인칭 3D 농장 시뮬레이션 게임
- 밭 갈기, 씨앗 심기, 물 주기, 수확 및 판매를 통해 탈출 비용 마련
- 여러 플레이어가 같은 월드에 접속하여 밭과 재화를 같이 관리하고, 서로의 이동과 도구 사용을 확인할 수 있도록 확장
- 서버: 단일 서버에서 월드, 플레이어, 작물, 인벤토리, 재화, 탈출 상태 관리 예정
- 서버–클라이언트 통신: TCP 적용 예정
- 개발 언어 및 그래픽스 API: C++ / OpenGL
- 작업 IDE: Microsoft Visual Studio
- 버전 관리: Git

2. 애플리케이션 기획

2.1 게임 진행 방식 및 월드 구성

- 외딴 섬에서 작물을 재배하고 판매하여 자금을 모은 뒤, 섬을 탈출하는 것을 목표로 하는 3D 농장 시뮬레이션 게임.
- 씨앗 구매 → 밭 갈기 → 씨앗 심기 → 물 주기 → 수확 → 판매의 과정을 반복하며 탈출 비용 마련.
- 월드는 밭, 작물별 상점, 탈출용 보트와 배경 오브젝트로 구성되며, 외부 배치 데이터를 읽어 각 객체의 종류와 위치 설정.
- 단일 서버가 하나의 월드를 관리하며, 별도 로비 없이 접속한 플레이어들이 동일한 월드에서 플레이하도록 구현할 예정.
- 플레이어 입장 시 객체를 생성하고 현재 월드 상태를 전달하며, 퇴장 시 해당 객체 삭제. 플레이 기록은 별도로 저장하지 않을 예정.

2.2 플레이어 이동 및 조작

- WASD를 통해 전후좌우 이동, SPACE를 통해 점프.
- 마우스 이동에 따라 3인칭 카메라가 회전하며, 카메라 방향을 기준으로 플레이어

의 이동 방향과 몸의 방향 결정.

- 클라이언트에서 충돌 검사로 이동 가능한 위치를 계산하고, 수직 속도와 중력을 적용하여 점프와 착지 처리. 계산한 결과는 자신의 캐릭터와 카메라에 즉시 반영.
- 변경된 플레이어의 위치와 방향을 서버에 전송하고, 서버는 해당 정보를 저장하여 다른 클라이언트에 전달할 예정.
- 각 클라이언트는 전달받은 정보를 해당 플레이어 객체에 적용하여 상대 플레이어의 움직임 동기화.



그림 1. 플레이어의 3 인칭 시점

2.3 오브젝트 상호작용

- 밭, 상점, 보트 등 상호작용 가능한 객체에 접근한 뒤 E를 눌러 해당 객체의 기능 사용.
- 객체별로 정해진 상호작용 거리 안에서 가장 가까운 대상을 선택하며, 대상이 선택되면 화면에 상호작용 안내 표시.
- 상호작용 객체는 공통 인터페이스인 IInteractable을 사용하며, 같은 E 입력에도 대상에 따라 농사, 상점 이용, 탈출 등 서로 다른 동작 수행.
- 클라이언트가 상호작용을 요청하면 서버에서 플레이어의 위치, 장착한 아이템과 대상의 상태를 확인하여 실행 가능 여부 판정.
- 서버의 처리 결과를 전달받은 클라이언트는 변경된 객체 상태와 UI, 필요한 시각 효과를 반영하도록 구현할 예정.

2.4 인벤토리 및 도구 선택



그림 2. 인벤토리 및 단축키

- 인벤토리는 당근·양배추 씨앗과 수확물의 보유 수량을 관리하며, 화면 하단에 아이템별 수량 표시.
- 숫자 키 1~5를 통해 밭갈이 도구, 물뿌리개, 낫, 당근 씨앗, 양배추 씨앗 선택.
- 아이템 종류별 보유 수량과 현재 장착한 아이템을 구분하여 관리하며, 선택한 농기구는 캐릭터의 손에 해당 모델 표시.
- 장비 변경 시 선택한 아이템 정보를 서버에 전달하고, 서버가 이를 다른 클라이언트에도 전달하여 서로의 장착 도구를 확인할 수 있도록 구현할 예정.
- 씨앗 사용과 수확물 획득 등 아이템 수량이 변경되는 동작은 서버에서 처리하고, 결과에 따라 클라이언트의 인벤토리 정보 갱신.

2.5 농사 및 작물 성장

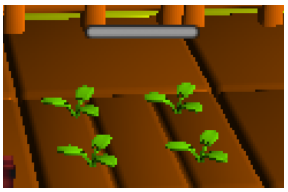
		
새싹	중간 성장	성숙(수확 가능)

그림 3. 양배추 성장 단계

- 재배 가능한 작물은 당근과 양배추이며, 밭 갈기 → 씨앗 심기 → 물 주기 → 성장 → 수확의 순서로 농사 진행.
- 빈 땅에 밭갈이 도구를 사용하면 경작 가능한 상태로 변경되며, 해당 밭에 씨앗을 사용하면 씨앗 수량을 하나 차감하고 작물 생성.
- 작물은 새싹, 중간 성장, 성숙의 세 단계 모델로 구성. 단계별로 필요한 물이 확보된 상태에서 일정 시간이 지나면 다음 단계로 성장하며 모델 변경.
- 물뿌리개 사용 시 작물에 물을 추가하고 물 파티클과 캐릭터의 팔 동작 재생.
- 작물을 수확하면 밭이 경작 전 상태로 돌아가며, 다음 작물을 심기 위해서는 밭갈기부터 다시 진행하도록 변경할 예정.

- 서버에서 수확물 지급과 밭의 상태 변경을 함께 처리하고, 결과를 전달받은 클라이언트는 인벤토리와 밭의 표시 상태 갱신.
- 여러 플레이어가 같은 밭에 작업을 요청하는 경우 서버가 현재 상태를 기준으로 순서대로 판정하여 중복 파종이나 중복 수확 방지.

2.6 상점 및 재화 관리

- 당근과 양배추에 대응하는 상점에서 씨앗을 구매하거나 해당 작물의 수확물 판매.
- 상점에 접근하여 E를 누르면 거래 UI가 표시되며, B로 씨앗 구매, S로 작물 판매, X로 상점 이용 종료.
- 구매 시 필요한 금액을 차감하고 씨앗을 추가하며, 판매 시 보유한 수확물을 차감하고 판매 금액 지급.
- 작물별 구매·판매 가격은 외부 데이터로 관리.
- 클라이언트가 구매·판매를 요청하면 서버가 잔액과 보유 수량을 확인한 뒤 아이템과 재화의 변경을 함께 처리할 예정.
- 동시에 발생하는 거래에서도 같은 재화나 아이템이 중복 사용되지 않도록 서버에서 처리하고, 결과를 관련 클라이언트에 전달하여 UI 갱신.

2.7 탈출 및 게임 종료

<table>
<tr>
<td> 
</td>
<td> 
</td>
</tr>
<tr>
<td>탈출 조건 미충족 안내</td>
<td>탈출 성공 장면</td>
</tr>
</table>

그림 4. 탈출 조건 안내와 엔딩 화면

- 농사와 판매를 통해 목표 금액인 10,000G를 마련한 뒤, 보트와 상호작용하여 섬 탈출.
- 목표 금액이 부족하면 추가로 필요한 금액을 안내하며, 조건을 충족하면 탈출 여부를 확인하는 UI 표시.

- Y를 선택하면 탈출 장면으로 전환하고, N을 선택하면 게임 진행.
- 탈출 장면에서는 보트 이동과 종료 문구를 표시한 뒤 화면을 어둡게 처리하며 게임 종료.
- 네트워크 적용 시 서버에서 재화와 탈출 조건을 확인하고 결과를 전달하며, 탈출 결과를 받은 클라이언트가 종료 장면으로 전환하도록 구현할 예정.

2.8 까마귀(추가요소)

- 맵 전체에 2~3마리의 까마귀가 날아다니며 농사를 방해하는 NPC로 추가할 예정.
- 까마귀는 작물이 있는 밭으로 날아와 작물을 쪼아 먹으며, 플레이어가 일정 거리 안으로 접근하면 날아가도록 구현할 예정.
- 저녁 시간대에는 낮보다 도주 판정 거리를 짧게 설정하여, 플레이어가 아주 가까이 접근해야 날아가도록 행동 조건 변경.
- 서버에서 까마귀의 위치와 행동, 플레이어와의 거리 및 시간대에 따른 도주 여부를 판정하고, 변경된 상태를 각 클라이언트에 전달하여 동기화할 예정.

2.9 주요 게임 화면



그림 5. 게임 시작 화면



그림 6. 게임 플레이 화면



그림 7. 밭에 물 뿌리는 장면

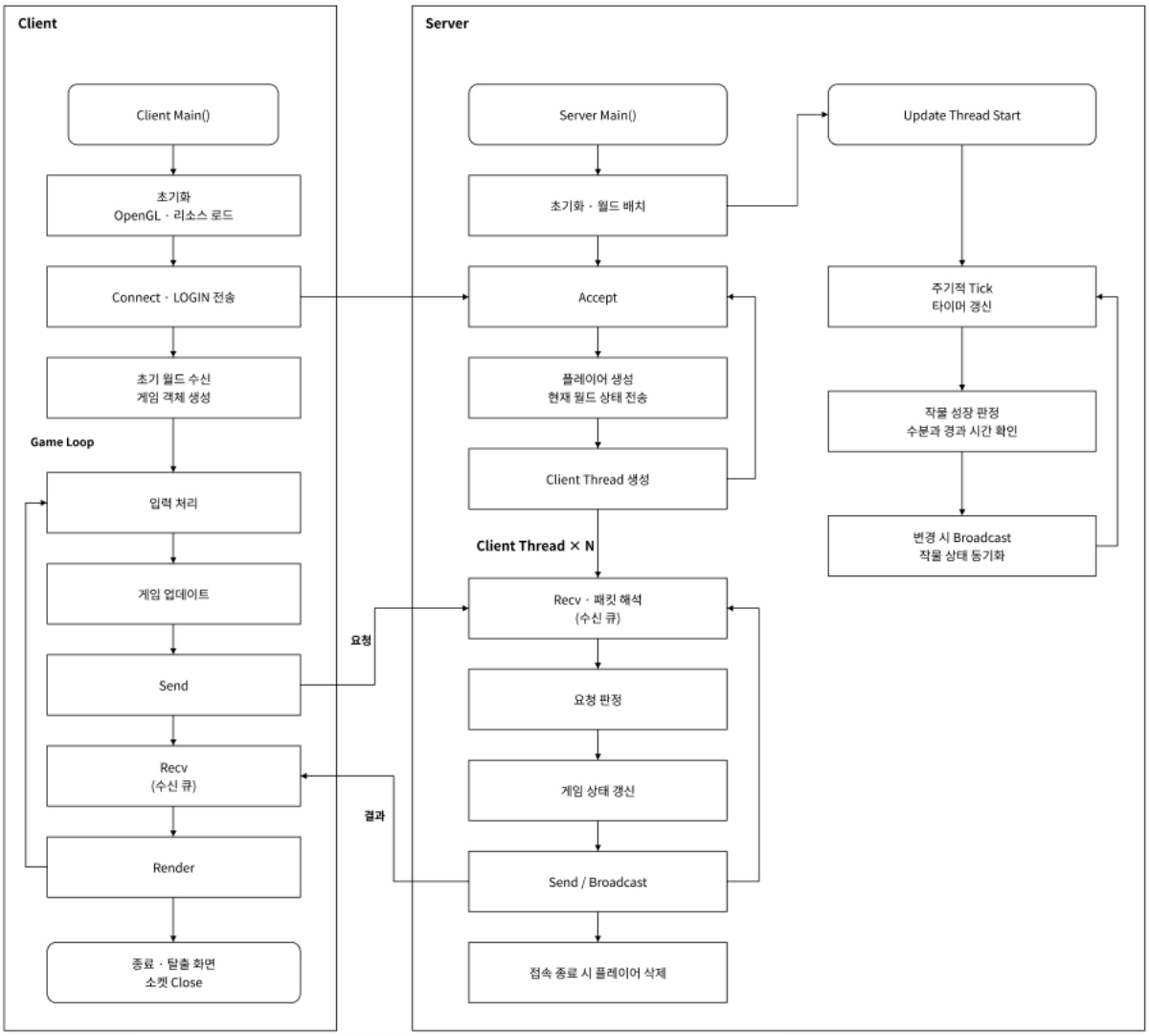


그림 8. 상점 UI 장면

3. High-Level Design

3.1 프로그램 흐름

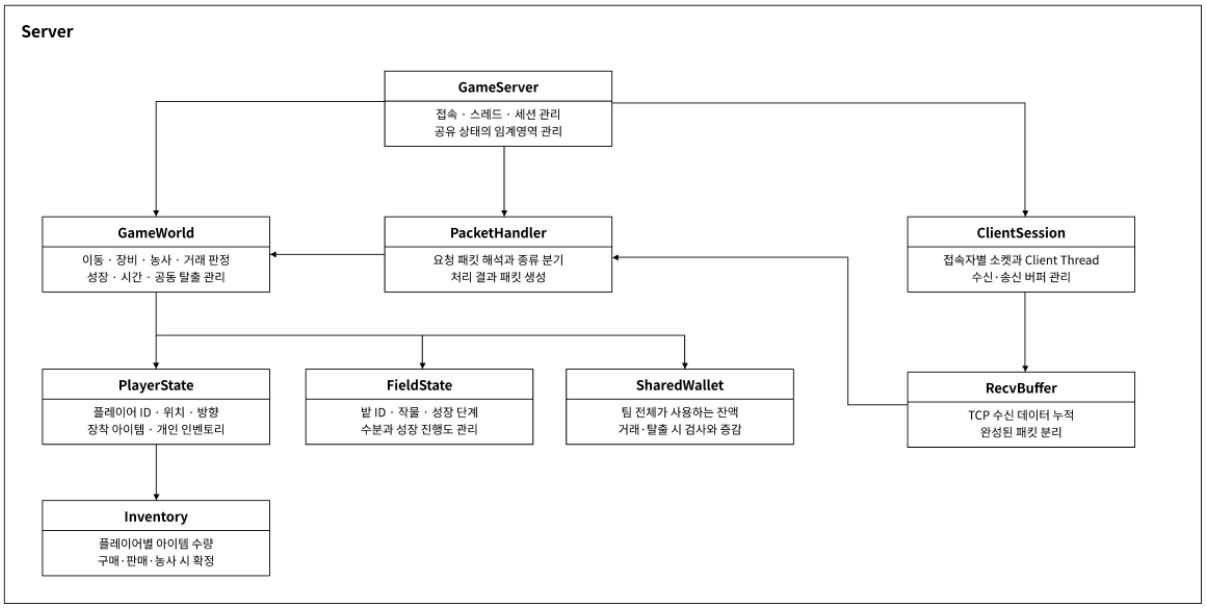
프로그램 흐름

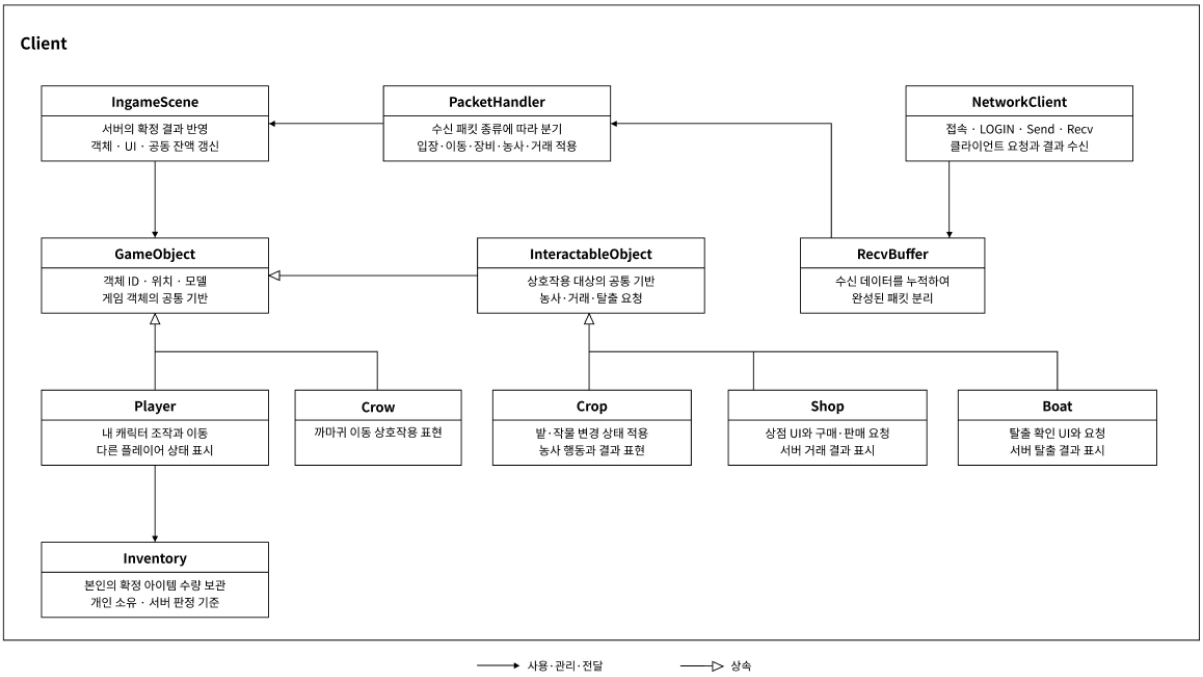


4. Low-Level Design

4.1 클래스 설계

클래스 개요





4.2 프로토콜 설계

Client to Server

패킷	담을 정보
C2S_LOGIN_PACKET	플레이어 생성 요청
C2S_MOVE_OBJECT_PACKET	내 위치(x, y, z), 몸 방향, 이동·점프 상태
C2S_EQUIP_PACKET	선택한 아이템 ID
C2S_INTERACT_FIELD_PACKET	특정 밭과 상호작용 요청
C2S_SHOP_BUY_PACKET	작물 구매 요청
C2S_SHOP_SELL_PACKET	작물 판매 요청
C2S_BOAT_ESCAPE_PACKET	탈출 요청

Server to Client

패킷	담을 정보
S2C_LOGIN_INFO_PACKET	플레이어 초기화 및 생성
S2C_ADD_OBJECT_PACKET	월드에 오브젝트 생성
S2C_REMOVE_OBJECT_PACKET	제거할 오브젝트
S2C_MOVE_OBJECT_PACKET	월드 오브젝트 위치 동기화
S2C_EQUIP_OBJECT_PACKET	플레이어 ID, 장착 아이템 ID
S2C_FIELD_UPDATE_PACKET	밭과 상호작용
S2C_SHOP_BUY_RESULT_PACKET	작물 구매
S2C_SHOP_SELL_RESULT_PACKET	작물 판매
S2C_TIME_SYNC_PACKET	현재 게임 시간, 시간 진행 속도
S2C_ESCAPE_RESULT_PACKET	탈출 결과 반환

5. 역할 분담 및 개발 일정

5.1 팀원 별 역할 분담

정태연	김성찬	김정민
플레이어 조작 및 클라이언트 서버 접속 기능 구현 서버 플레이어 상태·게임 시간 관리 및 탈출 기능 구현 로그인·이동·시간 동기화·탈출 관련 프로토콜 구현	클라이언트 상호작용·인벤토리·농사·상점 기능 구현 서버 인벤토리·밭 상태·공유 재화 관리 및 판정 구현 장비 변경·농사·구매·판매 관련 프로토콜 구현	서버·클라이언트 네트워크 프레임워크 및 스레드 동기화 구현 월드 객체 관리·배치 데이터 로딩 및 까마귀 기능 구현 객체 생성·삭제·위치 동기화 관련 프로토콜 구현

5.2 개발 일정

※ 날짜별로 팀원의 담당 작업을 정리했습니다. `C2S_/S2C_` 표기는 프로토콜 패킷입니다. 일요일은 빨간색, 토요일은 파란색으로 표시했습니다.

날짜	정태연	김성찬	김정민
11/1 (일)	• 클라이언트 NetworkClient (Winsock connect)	–	• GitHub 생성, 서버 프로젝트 생성 (Winsock, listen) • Accept, ClientSession
11/2 (월)	–	• Crop·Inventory·Shop 공용 로직 분리	• 서버 RecvBuffer, PacketHandler
11/3 (화)	–	–	–
11/4 (수)	• 클라이언트 RecvBuffer, PacketHandler	• crops.csv 로딩 구현	–
11/5 (목)	• 타이틀 화면에서 서버 접속 후 인게임 전환	–	–
11/6 (금)	–	• 서버 Inventory 구현	• Send 구현, GameWorld 뼈대 잡기
11/7 (토)	• 서버 PlayerState 클래스	• 서버 FieldState 구현	• Update Thread, 임계영역
11/8 (일)	• **`C2S_LOGIN`** 처리, **`S2C_LOGIN_INFO`**	–	• 서버에서 월드 배치 데이터 로딩 후 렌더링
11/9 (월)	–	• **`C2S_EQUIP`** 전송 및 서버 장비 저장	• **`S2C_ADD_OBJECT`** (밭, 상점, 보트)
11/10 (화)	• **`C2S_MOVE_OBJECT`**	–	• 클라이언트 IngameScene 객체를 패킷 기준으로 생성

날짜	정태연	김성찬	김정민
11/11 (수)	• **S2C_MOVE_OBJECT**	• **S2C_FIELD_UPDATE** 및 Crop 상태 반영	• **S2C_REMOVE_OBJECT**, 접속 종료 처리
11/12 (목)	• 다른 플레이어 객체	–	–
11/13 (금)	–	• **S2C_EQUIP_OBJECT** 및 상대 손 도구 표시	• 후에 접속한 플레이어에게 현재 월드 상태 전송
11/14 (토)	–	• **C2S_INTERACT_FIELD** 요청 전송	–
11/15 (일)	• 다른 플레이어 이동 보간, 걷기·점프 애니메이션 반영	–	–
11/16 (월)	–	• 서버 밭갈이·파종 판정	• 까마귀 모델 준비
11/17 (화)	• 서버 TimeSystem 이식	–	• 서버 Crow 객체 생성 (ID, 위치)
11/18 (수)	–	• 파종·수확 인벤토리 수량 동기화	• 클라이언트 Crow 모델 불러오기
11/19 (목)	• **S2C_TIME_SYNC**	–	–
11/20 (금)	–	• 밭 상호작용 거부 결과 전송·안내	–
11/21 (토)	–	• 서버 물주기·성장 판정	• Crow 배회·밭 접근 이동
11/22 (일)	• 클라이언트 재화 UI를 서버 값으로 표시	–	• Crow 도주 판정 (거리, 낮/밤 거리 차이)
11/23 (월)	–	• 서버 수확 판정 및 수확물 지급	–
11/24 (화)	–	–	• Crow 2~3마리 다중 생성
11/25 (수)	–	• 다른 플레이어 물주기 파티클·도구 모션 동기화	• Crow 위치 동기화 (**S2C_MOVE_OBJECT**)
11/26 (목)	• **C2S_BOAT_ESCAPE**	–	–
11/27 (금)	–	• 동시 거래 처리 보호 구현 • Inventory·SharedWallet 변경을 같은 보호 범위에서 처리	–
11/28 (토)	• 서버 탈출 조건 (10,000G) 판정	–	• Crow 날개 애니메이션
11/29 (일)	• **S2C_ESCAPE_RESULT**	–	• Crow 날개 애니메이션

날짜	정태연	김성찬	김정민
11/30 (월)	–	• **S2C_SHOP_BUY / SELL_RESULT** 클라이언트 UI 반영	• Crow 쪼아 먹기, 까마귀에게 먹힌 작물을 밭 상태로 복귀
12/1 (화)	–	–	• Crow 쪼아먹는 모션
12/2 (수)	• 클라이언트 ExitScene 연동, 전체 클라이언트 탈출 종료	–	–
12/3 (목)	–	–	–
12/4 (금)	–	• **C2S_SHOP_BUY** 서버 판정	–
12/5 (토)	• 접속 실패·서버 끊김 처리, 탈출 후 서버 상태 정리	• **C2S_SHOP_SELL** 서버 판정	–
12/6 (일)	–	–	• 연결 끊김 시 타이틀 화면 복귀
12/7 (월)	–	–	–
12/8 (화)	–	–	–
12/9 (수)	**최종 제출일**		